Launch readiness: security controls, AI risks and website quality

One launch review, several scopes

The sources contribute a general 20-point launch list, an advanced security installment numbered 55-70, and a security checklist researched against OWASP. The numbering is the creator's organization; the missing earlier installment is not available evidence. Neither a checklist length nor an AI-generated audit establishes that an application is secure.

Use the OWASP Top Ten (https://owasp.org/www-project-top-ten/) to identify broad risks, ASVS (https://owasp.org/www-project-application-security-verification-standard/) to select specific applicable requirements, and the LLM guidance (https://genai.owasp.org/llm-top-10/) for model-facing features. These serve different purposes. Test only applications and fixtures you own or are permitted to assess.

Gate 1: access and integrity

For every protected resource, test an unauthenticated caller, the owner, another user and a restricted role. Include bulk, export, realtime and GraphQL routes. Check password/session reset, logout invalidation, CSRF where applicable and server-side validation. Browser-hidden controls are not authorization. Exercise duplicate and replayed webhooks and monetary/business-rule edge cases.

Gate 2: agent and supply-chain boundaries

Inventory tools and scopes. Keep retrieved text as data; a document cannot grant tool permission. Validate arguments, require the appropriate human decision for consequential actions and retain deterministic limits outside the model. Check secret handling, CI token permissions, pinned dependencies where appropriate and protected deployment environments. GitHub's Actions hardening guidance (https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions) is a concrete starting point.

Gate 3: failures and recovery

Test timeouts, provider outages, stale data, retry duplication, partial writes and rollback. A failed risk check should not silently grant access. Add backup restoration and logging checks without leaking credentials or private payloads. Use Juice Shop (https://owasp.org/www-project-juice-shop/) as an intentionally vulnerable training environment, not as a production dependency.

Gate 4: a usable public site

The broader launch reel adds keyboard navigation, focus visibility, accessible labels, contrast, real 404 behavior, mobile layouts, image budgets, metadata and working links. Lighthouse (https://developer.chrome.com/docs/lighthouse/overview/) and axe (https://www.deque.com/axe/devtools/) help gather evidence but do not replace manual keyboard and task testing. Privacy and legal requirements depend on the actual data and jurisdiction; generic generated legal pages are not an assessment.

Evidence workbook project

Create rows with control, applicability, test fixture, expected outcome, observed outcome, evidence, fix, owner and retest date. Begin with five high-impact controls for one small app. Keep "not tested" distinct from "passed" and "not applicable." Export a reviewable report with the tested commit and environment.

```
Build a launch evidence workbook for a small application. Select applicable
ASVS and agent-risk checks. Add authorized fixture-based tests, evidence links,
remediation and retest status. Include accessibility and failure recovery.
Do not mark a control passed unless its expected result was observed.
```

All three original checklists remain available for detailed reference. The unified workflow is the review order, not a claim that every control has already been tested on your app.

Original sources

- https://www.instagram.com/reel/DdKhqxHvjIP/
- https://www.instagram.com/reel/DcbDo37zTvJ/
- https://www.instagram.com/reel/DeM46Iay29L/